

# FEKO 多点电磁仿真全流程

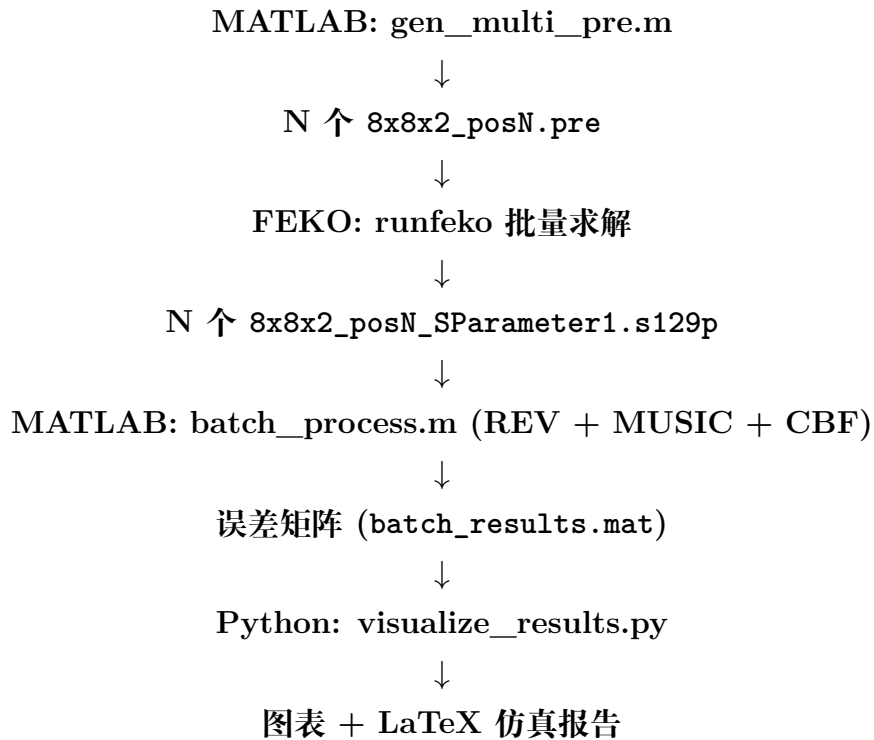
2026 年 7 月 29 日

## 1 总体架构

本流程实现了一套端到端的 FEKO 电磁仿真自动化方案，包含三个层级：

1. **FEKO 层**：CADFEKO 建模 + EDITFEKO 脚本 + 命令行批量求解
2. **MATLAB 层**：.pre 文件自动生成 + S 参数读取 + REV 相位恢复 + 3D 定位
3. **Python 层**：结果可视化 (热力图/3D 散点/统计图)

三层的衔接关系如：



## 2 第一步: CADFEKO 建模

### 2.1 几何体创建

1. 新建 CADFEKO 项目, 创建  $10 \times 8 \times 3$  m 长方体房间 (6 个 Rectangle)
2. 创建参考偶极子天线 `dipole` (细长圆柱, 端口 `Port1`), 放在房间外
3. 创建发射偶极子天线 `source_dipole` (细长圆柱, 端口 `SPort1`), 放在  $[5, 4, 1.5]$
4. 两偶极子均平行于 XOY 平面 (水平极化)

### 2.2 材质与网格

1. 在介质库创建混凝土介质: 相对介电常数 5.5, 损耗正切 0.2
2. 创建涂层 `Concrete_Wall`: 厚度 0.3 m, 基于混凝土介质
3. 将涂层应用到 6 个 Rectangle 的面
4. Create Mesh: 天线细网格, 墙面粗网格 (UTD 用多边形近似)

### 2.3 求解设置

- 频率: 2.4 GHz
- UTD: 启用, 最大反射次数 = 1
- Skin effect: `Concrete_Wall` 涂层应用到 6 面墙

### 2.4 导出

CADFEKO 保存后自动生成 `8x8x2.cfm` 和 `8x8x2.pre`。

**注意:** 此后 不再使用 CADFEKO 打开该项目, 否则它会覆盖手动编辑的 `.pre` 文件。所有后续修改都在 `.pre` 文本中进行。

## 3 第二步: EDITFEKO 脚本编辑

打开 `8x8x2.pre`, 在其基础上补入三块内容。

### 3.1 阵列 TG 复制循环

```

** Generate the array
!!for #i=1 to 128
#x=fileread("two_arrays_8x8.inc",#i+1,1)+1
#y=fileread("two_arrays_8x8.inc",#i+1,2)
#z=fileread("two_arrays_8x8.inc",#i+1,3)
TG:1 : dipole.Wire1 : dipole.Wire1 : #i : 2 : : : : #x : #y : #z
TG:1 : dipole.Wire1.Port1 : dipole.Wire1.Port1 : #i : 2 : : : : #x : #y : #z
!!next

```

从 two\_arrays\_8x8.inc 读取 128 个阵元坐标, 逐个平移复制参考偶极子。

### 3.2 发射源 TG (批量仿真用)

```

** Move source to target position
#sx = -3.000000
#sy = -2.000000
#sz = -1.000000
TG:1 : source_dipole.SWire1 : source_dipole.SWire1 : 99 : 2 : : : : #sx : #sy :
      #sz
TG:1 : source_dipole.SWire1.SPort1 : source_dipole.SWire1.SPort1 : 99 : 2 : ...

```

TG 索引固定为 99 (避开 PREFEKO 变量 #i 冲突)。参考点 [5, 4, 1.5] 时跳过 TG, 直接使用原端口。

### 3.3 S 参数配置

```

** SParameterConfiguration1
A1: 0 : source_dipole.SWire1.SPort100 : 0 : : : : 1 : 0 : : : : 50
!!for #i=2 to 129
A1: 1 : dipole.Wire1.Port#i : 0 : : : : 0 : 0 : : : : 50
!!next
SP: 1 : 1 ** SParameter1

```

129 端口 S 参数: Port1 = 发射 (50Ω 激励), Port2–129 = 128 接收阵元 (50Ω 负载)。

## 4 第三步: FEKO 命令行求解

单点求解命令:

```
D:\Software\FEKO2022\feko\bin\runfeko.exe 8x8x2.pre
```

PREFEKO 读取.pre + .cfm → 生成.fek → 求解器运行 → 输出.s129p。

## 5 第四步：多点批量自动化

### 5.1 MATLAB: gen\_multi\_pre.m

定义源位置网格 (余弦间距, 边角密集):

```
x_list = 0.5 + 4.5*(1-cos(linspace(0,pi,4))); % 0.5, 2.75, 7.25, 9.5
y_list = 0.5 + 3.5*(1-cos(linspace(0,pi,5))); % 0.5, 1.53, 4.0, 6.47, 7.5
z_list = 0.3 + 1.2*(1-cos(linspace(0,pi,5))); % 0.3, 0.65, 1.5, 2.35, 2.7
```

网格生成: meshgrid(x\_list, y\_list, z\_list) → 100 个  $[x, y, z]$  坐标。

对每个位置:

- 计算 TG 平移量  $\Delta x = tx - 5, \Delta y = ty - 4, \Delta z = tz - 1.5$
- 若平移量  $\approx 0$  (参考点): 跳过 TG, 端口标号用 SPort1
- 若非参考点: 写入 TG 平移命令, 端口标号用 SPort100
- 生成独立 8x8x2\_posN.pre 文件

同时生成 run\_all.bat:

```
@echo off
echo [time] 位置 1/100: (x,y,z)
D:\Software\FEK02022\feko\bin\runfeko.exe 8x8x2_pos1.pre
if %errorlevel% neq 0 echo ERROR & pause
... (重复 100 次)
```

### 5.2 终端执行

```
del 8x8x2_pos*.pre; del 8x8x2_pos*.fek
matlab -r "gen_multi_pre" % 生成 .pre + run_all.bat
.\run_all.bat % 批量运行 (~3小时)
```

每条命令的输出: PREFEKO 生成.fek (910 天线 +6 多边形板) → 求解器 (MoM + UTD) → .s129p 文件。

## 6 第五步: MATLAB 后处理

### 6.1 batch\_process.m 执行流程

1. 加载 source\_positions.mat (100 个源坐标)
2. 从 two\_arrays\_8x8.inc 和 8x8x2.out 读取阵列坐标
3. 对每个位置  $i = 1 \rightarrow 100$ :
  - (a) 读 8x8x2\_posi\_SParameter1.s129p
  - (b) 提取 129 端口 S 参数, 分离两个 64 元阵列
  - (c) REV 三功率计算 ( $0^\circ/90^\circ/180^\circ$  逐阵元相移)
  - (d) REV 公式解算复数响应  $k_m e^{jX_m}$
  - (e) 3D MUSIC: 噪声子空间投影, 谱峰搜索 ( $101 \times 81 \times 31$  点)
  - (f) 3D CBF: 常规波束形成, 谱峰搜索 (同网格)
  - (g) 记录估计位置和误差
4. 汇总统计: 均值/中位/最值, 逐点误差表
5. 保存 batch\_results.mat, 导出 results\_summary.txt
6. 生成分层热力图 (MUSIC vs CBF)

## 7 第六步: Python 可视化

output/visualize\_results.py 读取 batch\_results.mat, 生成 5 张图:

| 图号   | 内容                            |
|------|-------------------------------|
| fig1 | 逐 z 层热力图矩阵 (5 层, 插值平滑, 含阵列标记) |
| fig2 | 3D 空间散点图 (误差色标, 阵列位置)         |
| fig3 | 统计面板: 直方图 + CDF 累积分布          |
| fig4 | 误差 vs. 到阵列距离 (按 z 着色)         |
| fig5 | 四象限综合信息图 (热力图 + 箱线图 + 统计摘要)   |

运行:

```
pip install scipy matplotlib
python visualize_results.py
```

## 8 第七步：撰写仿真记录

output/simulation\_record.tex 为 LaTeX 报告模板，引用生成的 5 张图，包含：

- 系统参数表
- 全局 + 分层统计
- 空间分布规律分析
- 与 MATLAB 理想仿真的对比讨论
- 结论与实际部署建议

编译：

```
xelatex simulation_record.tex
xelatex simulation_record.tex % 跑两次处理交叉引用
```

## 9 关键技术细节

### 9.1 REV 算法

由三组合成功率  $\{P_0, P_{90}, P_{180}\}$  解线性方程组：

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & -j & j \\ 1 & -1 & -1 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} P_0 \\ P_{90} \\ P_{180} \end{bmatrix} \implies \Delta\phi = \arg(x_2), \quad E_{mr} = \frac{E_m}{E_m + \bar{E}_m e^{j\Delta\phi}}$$

### 9.2 3D MUSIC

$$P_{\text{MUSIC}}(\mathbf{p}) = \frac{1}{\mathbf{a}^H(\mathbf{p}) \mathbf{U}_n \mathbf{U}_n^H \mathbf{a}(\mathbf{p})}, \quad \mathbf{a}(\mathbf{p}) = \exp\left(-j \frac{2\pi}{\lambda} \|\mathbf{p} - \mathbf{r}_i\|\right)$$

### 9.3 3D CBF

$$P_{\text{CBF}}(\mathbf{p}) = |\mathbf{a}^H(\mathbf{p}) \mathbf{x}|^2$$

## 9.4 端口编号规则

- 阵列端口: TG 索引  $\#i$  创建端口号  $\#i + 1$  (原 Port1 已存在), 引用 Port $\#i$
- 发射端口: TG 索引 99 创建端口号 100, 引用 SPort100; 参考点直接引用 SPort1
- S 参数矩阵: 行 = 129 端口, 列 1 = 发射到各接收端口的传输系数

## 9.5 避坑指南

1. **CADFEKO 与 EDITFEKO 互斥**: 一旦手动编辑.pre, 不要再打开 CADFEKO (会锁定设置或覆盖.pre)
2. **网格密度**: 墙面用 UTD 粗网格 (多边板), 不能生成 98 万三角形 (否则 MoM 矩阵  $> 16$  TB)
3. **端口标签**: TG 复制后端口号 = 原号 + TG 索引, 不是简单替换
4. **PREFEKO 变量**: 标签中的 # 被解释为变量引用, 用 ## 转义, 或避免在标签中使用
5. **参考点重叠**: 当目标 = CADFEKO 中的原始天线位置时, 不能 TG (会导致线段重叠), 需直接引用原端口

# 10 省流版运行流程

1. 清理旧文件: `del 8x8x2_pos*.pre; del 8x8x2_pos*.fek`
2. MATLAB 生成.pre: `gen_multi_pre`
3. 终端批量 FEKO: `.\run_all.bat`
4. MATLAB 批量分析: `batch_process`
5. Python 可视化: `cd output && python visualize_results.py`